

Аннотация

Работа посвящена разработке системы автоматизированной проверки задач, решаемых на языках программирования. В аналитической части рассмотрены три подхода к оценке решений учащихся: сравнение вывода программы с эталонным ответом, оценка с применением больших языковых моделей и семантический анализ структуры кода. Показано, что методы на основе LLM обладают недостаточной точностью для применения в образовательном процессе.

Разработанная система построена на микросервисной архитектуре и включает сервис фронтенда на React, бэкенд на Rust и сервис раннера, выполняющий пользовательский код в изолированных Docker-контейнерах. Взаимодействие между компонентами организовано через очередь сообщений PGMQ на основе PostgreSQL. В раннере реализован метод проверки путём сравнения вывода с эталонным ответом и семантический анализ на уровне структуры кода. Система поддерживает несколько языков программирования, разграничение прав доступа для трёх ролей пользователей и детальный отчёт по результатам тестирования.

СОДЕРЖАНИЕ

Перечень сокращений и обозначений	3
Введение	4
1 Анализ предметной области и существующих решений	5
1.1 Поиск методов для оценки решений учащихся	5
1.2 Проблемы метода сравнения ответа с эталонным решением	6
1.3 Использование искусственного интеллекта для оценки решения	7
1.4 Семантический анализ решения	8
2 Проектирование системы автоматизированной проверки задач	11
2.1 Выбор технологий для разработки системы	12
2.2 Поиск способов безопасного исполнения пользовательского кода	13
2.3 Бизнес процесс проверки решения	14
3 Разработка системы автоматизированной проверки задач	16
3.1 Разработка схемы базы данных	16
3.2 Разработка фронтенда	19
3.3 Разработка бэкенда	22
3.4 Разработка раннера решений	24
3.5 Тестирование	26
Заключение	28
Список использованных источников	29

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API Application Programming Interface

AST Abstract Syntax Tree

HTTP HyperText Transfer Protocol

JSON JavaScript Object Notation

JWT JSON Web Token

LLM Large Language Model

OOM Out Of Memory

PGMQ PostgreSQL Message Queue

REST Representational State Transfer

SQL Structured Query Language

UI User Interface

ВВЕДЕНИЕ

Задачи по программированию занимают центральное место в подготовке специалистов в области информационных технологий. Их ручная проверка при большом потоке студентов превращается в узкое место учебного процесса: преподаватель тратит значительное время на рутинные операции и физически не успевает давать развёрнутую обратную связь каждому обучающемуся. Автоматизация проверки позволяет получить результат уже через несколько секунд после отправки решения и снять нагрузку с преподавателя.

Существующие платформы — Codeforces, LeetCode, Яндекс.Контест — создавались в первую очередь для соревновательного программирования. Развернуть их в рамках учебного заведения и настроить под конкретный курс затруднительно: они не предусматривают гибкой настройки критериев проверки, слабо интегрируются с внутренней инфраструктурой и, как правило, поддерживают лишь один метод оценки — сравнение вывода программы с эталоном. Создание специализированной системы, ориентированной на учебный процесс и допускающей несколько методов проверки, является актуальной задачей.

Целью данной выпускной квалификационной работы является разработка системы автоматизированной проверки задач, решаемых на языках программирования, которая поддерживает несколько методов оценки решений, безопасно исполняет пользовательский код в изолированной среде и предоставляет отдельные интерфейсы для обучающихся и преподавателей.

В ходе работы были рассмотрены методы оценки решений учащихся, из которых были выбраны наиболее подходящие для применения в образовательном процессе. На основе проведённого анализа была спроектирована архитектура автоматизированной системы для проверки задач, после чего была выполнена разработка данной системы.

1 Анализ предметной области и существующих решений

Прежде чем приступать к проектированию системы, необходимо разобраться, какие методы оценки решений задач по программированию существуют и в чём их особенности. При этом оценка конкретного решения от обучающегося не обязательно должна строиться на одном подходе: в зависимости от задачи итоговый результат может формироваться как на основе автоматического тестирования, так и с учётом анализа структуры кода или его оценки языковой моделью.

1.1 Поиск методов для оценки решений учащихся

В задаче автоматической проверки решений по программированию я выделил несколько основных методов оценки, каждый из которых имеет свои достоинства и недостатки.

Сравнение вывода программы с эталонным ответом. Система компилирует и запускает пользовательское решение с заранее подготовленными входными данными, а затем сравнивает полученный вывод с правильным ответом. Для каждой задачи преподаватель формирует набор тест-кейсов: каждый из них содержит входные данные и ожидаемый результат. Этот метод лежит в основе большинства существующих платформ — Codeforces, LeetCode, Яндекс.Контест — и прост в реализации, однако его недостаток состоит в полном отсутствии понимания структуры кода решения.

Оценка с применением больших языковых моделей (LLM). Вместо запуска программы модель анализирует текст исходного кода напрямую. Такой подход позволяет оценивать логику решения, его соответствие условию задачи и стиль написания, а также формировать текстовый комментарий для обучающегося [1]. Преимущество метода — возможность давать развёрнутую обратную связь даже по частично правильным решениям, без запуска кода.

Семантический анализ исходного кода. Код разбирается в структурированное представление — набор лексических токенов или абстрактное синтаксическое дерево (AST), — после чего к нему применяются заранее заданные преподавателем правила. Правила могут ограничивать класс допустимых решений: например, запрещать вложенные циклы или рекурсию. Данный метод подробно исследован в работе Федорова и Авксентьевой, где для представления кода предложено использовать семантические сети [2].

Каждый из перечисленных методов имеет свою область применения. Итоговая оценка решения обучающегося может учитывать результаты сразу нескольких подходов, что даёт преподавателю более полное представление о качестве работы студента.

1.2 Проблемы метода сравнения ответа с эталонным решением

Метод сравнения вывода с эталонным ответом работает на уровне входных и выходных данных и ничего не знает о том, как именно устроена программа внутри. Это порождает ряд практических проблем.

Первая из таких проблем это — алгоритмическая неэффективность остаётся незамеченной. Программа, решающая задачу с временной сложностью $O(n^2)$ вместо $O(n \log n)$, пройдёт все тесты точно так же, как и оптимальное решение, если временной лимит задан с запасом или объём тестовых данных невелик. В учебном контексте это означает, что студент может привыкнуть писать медленные решения, не осознавая этого.

Также данный метод может страдать от неполного покрытия входных данных. Набор тестов всегда конечен, и для достаточно сложных задач, как бы тщательно преподаватель их ни подготовил, всегда найдутся варианты входных данных, которые они не охватывают. Классические примеры: пустой массив, отрицательные числа, переполнение целочисленного типа, единственный элемент в коллекции. Программа проходит все имеющиеся тесты, но на граничных случаях работает неправильно.

Последняя проблема заключается в том что данный метод не оценивает понимание алгоритма. Для простых задач обучающийся может получить правильный вывод случайно или написать код, который работает только на конкретных значениях из готового набора тестов. Такое решение формально проходит проверку, хотя содержательно не является верным.

Перечисленные ограничения не исключают использования метода: он остаётся самым надёжным способом объективно подтвердить корректность работы программы. Однако в образовательном процессе их следует принимать во внимание при составлении тестов и формировании критериев оценивания.

1.3 Использование искусственного интеллекта для оценки решения

Применение языковых моделей для проверки студенческого кода — относительно молодое, но активно развивающееся направление исследований. В работе Мальцева, Назарова и Павловой [3] проводился эксперимент по использованию GPT-3.5 Turbo в роли бинарного классификатора решений задач на Python. Датасет включал 582 студенческих решения по четырём темам: условные операторы, математические функции и выражения, одномерные массивы и циклы while.

Модель получала формулировку задачи и исходный код решения и должна была ответить, верное ли это решение. Точность по темам варьировалась: от 0,73 для задач на математические функции и выражения до 0,90 для задач на одномерные массивы. Recall при этом оказался стабильно высоким — модель редко пропускала правильные решения. Однако precision для темы «Циклы while» составила лишь 0,63, то есть около трети решений, признанных моделью верными, содержали ошибки. Авторы делают вывод, что для ряда тем такой классификатор применим как вспомогательный инструмент в тренировочном режиме, но не подходит для итоговой аттестации.

В работе Карсы и Голдшмидта [1] рассматривалась уже численная оценка решений: модели давалось подробное описание критериев (рубрик оценивания), и она должна была выставить балл так же, как это сделал бы преподаватель. На материале экзаменов по C++ и Java лучший результат показала GPT-4o с коэффициентом корреляции от 0,6 до 0,9 в зависимости от конкретного набора задач. Было замечено, что GPT-3.5-Turbo систематически завышал оценки относительно преподавательских, тогда как GPT-4o — занижал.

Оба исследования демонстрируют, что языковые модели в принципе способны работать с кодом и давать оценки, частично совпадающие с мнением преподавателя. Однако конкретные числа дают иную картину: precision 0,63 у GPT-3.5 на теме «Циклы while» означает, что примерно каждая третья работа, признанная моделью верной, в действительности содержала ошибки. GPT-4o точнее, но её корреляция с преподавательскими оценками колебалась от 0,6 до 0,9 в зависимости от набора задач — и предсказать заранее, ближе к какой границе окажется результат на конкретной теме, не представляется возможным. Для учебного процесса такой разброс неприемлем: здесь ошибка оценки — это не просто неточность, а неправильная отметка в ведомости.

1.4 Семантический анализ решения

Семантический анализ предполагает разбор исходного кода в структурированное представление с последующим применением к нему заданных правил. Наиболее распространённый инструмент для этого — абстрактное синтаксическое дерево (AST): каждая конструкция языка (функция, цикл, условный оператор, выражение) становится узлом дерева, связанным с родительскими и дочерними конструкциями. Такое представление позволяет анализировать структуру программы независимо от конкретных имён переменных и форматирования.

В работе Федорова и Авксентьевой [2] предложен более детальный вариант: исходный код преобразуется в семантическую сеть, где узлами служат как конкретные программные элементы (переменные, выражения), так и концепты языка (тип «Цикл», тип «Условие»). Для двух программ вычисляется функция подобия, принимающая значения от 0 до 1, где 1 соответствует полному совпадению структуры. Авторы показали, что алгоритм уверенно разграничивает решения одной задачи (высокое подобие) от решений совершенно другой задачи (значение близко к нулю).

На рисунке 1 показано, как двустрочный фрагмент на Python — условный оператор с вызовом функции — превращается в AST. Корнем дерева служит узел `Module`, его единственным потомком является узел `If`, от которого расходятся две ветки. Левая, помеченная `condition`, описывает проверяемое условие: операция сравнения на равенство между переменной `paper` и строковой константой `"NeurIPS 2024"`. Правая ветка, `body`, представляет тело оператора: выражение-вызов функции `print` с аргументом `R2C2-Coder`. Такое структурированное представление позволяет анализировать программу независимо от её текстового оформления — применять правила непосредственно к типам узлов и их отношениям.

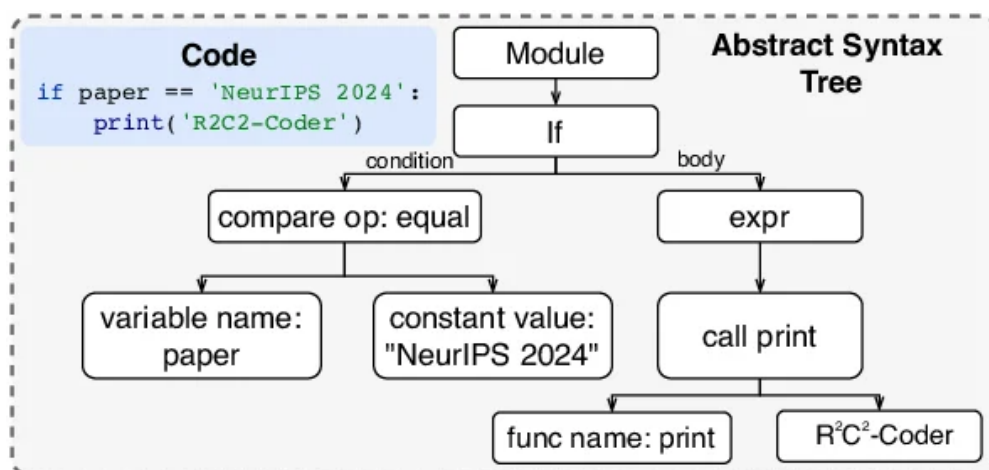


Рисунок 1 — Пример построения AST для условного оператора на Python

В контексте учебного процесса семантический анализ может применяться для задания ограничений на структуру решения. Преподаватель описывает правила, которым код должен удовлетворять. Например, запрет вложенных циклов вынуждает студента искать решение с линейной сложностью вместо написания тривиального $O(n^2)$ -алгоритма. Запрет рекурсии направляет к итеративному подходу. Требование использовать конкретную структуру данных помогает закрепить изучаемую тему на практике.

Технически обнаружение таких паттернов сводится к обходу AST: вложенные циклы можно найти, проверив наличие узла цикла внутри другого узла цикла; рекурсию — убедившись, что функция вызывает саму себя в своём теле. Основная сложность реализации состоит в том, что для каждого поддерживаемого языка программирования необходим отдельный парсер.

2 Проектирование системы автоматизированной проверки задач

Для реализации системы была выбрана микросервисная архитектура. Такой подход позволяет разделить задачи между независимыми компонентами, каждый из которых можно разрабатывать, тестировать и при необходимости масштабировать отдельно. Если в будущем нагрузка на проверку решений возрастёт, можно запустить дополнительные экземпляры сервиса раннера, не затрагивая остальные части системы.

Архитектура включает четыре компонента: сервис фронтенда, бэкенд, сервис раннера и базу данных. Бэкенд предоставляет HTTP API, которым пользуется фронтенд для отображения данных, управления контекстами и отправки решений. Раннер занимается выполнением пользовательского кода и записью результатов. Взаимодействие между бэкендом и раннером организовано через очередь сообщений.

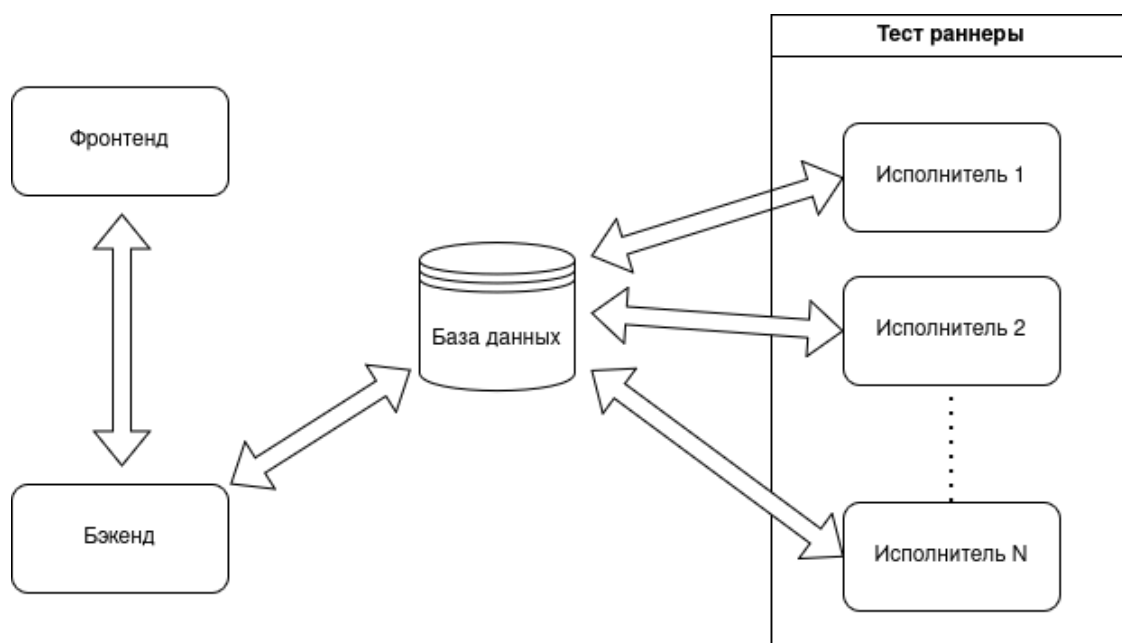


Рисунок 2 — Архитектура системы

На рисунке 2 показана общая схема взаимодействия компонентов. Стрелки от базы данных к раннерам отражают то, что задания на проверку раннер получает из базы данных.

Реализация методов оценки решений сосредоточена в сервисе раннера: он отвечает за запуск пользовательского кода и сравнение его вывода с эталонным ответом, а также за статический анализ структуры решения. Метод оценки на основе LLM в данной системе задействован не будет — как показал анализ актуальных исследований, точность языковых моделей при проверке кода остаётся недостаточной для образовательного процесса.

2.1 Выбор технологий для разработки системы

Фронтенд. Для разработки пользовательского интерфейса выбрана библиотека React. Выбор обусловлен её широкой распространённостью в веб-разработке: для React существует большое число готовых пакетов и решений, что существенно ускоряет разработку. Интерфейс реализован на TypeScript, что позволяет выявлять часть ошибок ещё на этапе компиляции.

Для получения данных от API используется библиотека SWR. В отличие от прямых вызовов `fetch`, SWR реализует стратегию `stale-while-revalidate`: компонент сначала отображает ранее закэшированные данные, а затем обновляет их в фоне. Это упрощает управление состоянием загрузки в компонентах. Операции изменения данных (отправка решения, создание контеста) выполняются через стандартный `fetch` с последующей инвалидацией кэша SWR.

Бэкенд. Для серверной части выбран язык программирования Rust. Помимо высокой производительности, компилятор Rust гарантирует отсутствие целого класса ошибок управления памятью, что подтверждается и в промышленных проектах: переход на Rust в экосистеме Android позволил значительно снизить долю уязвимостей, связанных с памятью

[4]. HTTP-сервер построен на фреймворке `axum`, а для асинхронного взаимодействия с PostgreSQL используется библиотека `sqlx`.

Очередь сообщений. Для передачи заданий между бэкендом и раннером используется расширение PostgreSQL под названием PGMQ. Оно реализует очередь сообщений непосредственно поверх таблиц PostgreSQL и не требует развёртывания отдельного брокера. В сравнении с RabbitMQ это заметно упрощает инфраструктуру: PostgreSQL уже используется в системе, поэтому добавление очереди не вносит новых сервисов для поддержки.

2.2 Поиск способов безопасного исполнения пользовательского кода

Запуск произвольного пользовательского кода на сервере создаёт несколько рисков. Программа может уйти в бесконечный цикл и заблокировать вычислительные ресурсы. Код на C++ способен поглотить всю доступную оперативную память. Вредоносная программа может попытаться прочитать файлы на диске, установить сетевое соединение или выполнить системные команды.

В качестве механизма изоляции пользовательского кода я рассмотрел два варианта: виртуальные машины и Docker-контейнеры.

Виртуальная машина обеспечивает полную аппаратную изоляцию за счёт эмуляции отдельного аппаратного окружения: гостевая операционная система не имеет прямого доступа к ресурсам хоста. Однако этот уровень изоляции достигается ценой значительных накладных расходов. Каждый экземпляр виртуальной машины требует выделения фиксированного объёма оперативной памяти, а время запуска измеряется секундами. При одновременной проверке множества решений это существенно ограничивает пропускную способность системы.

Docker-контейнеры используют то же ядро Linux, что и хост-система, и запускаются практически мгновенно. В работе Потдара и др. [5] проводилось сравнение производительности Docker-контейнеров и виртуальных машин

с использованием стандартных инструментов нагрузочного тестирования — Sysbench, Phoronix и Apache Benchmark. Результаты показали, что Docker обеспечивает существенно более высокую пропускную способность по памяти и производительность ввода-вывода, при этом по CPU-нагрузке контейнер практически не уступает нативному исполнению. Виртуальная машина во всех тестах показала заметно худшие результаты из-за накладных расходов гипервизора.

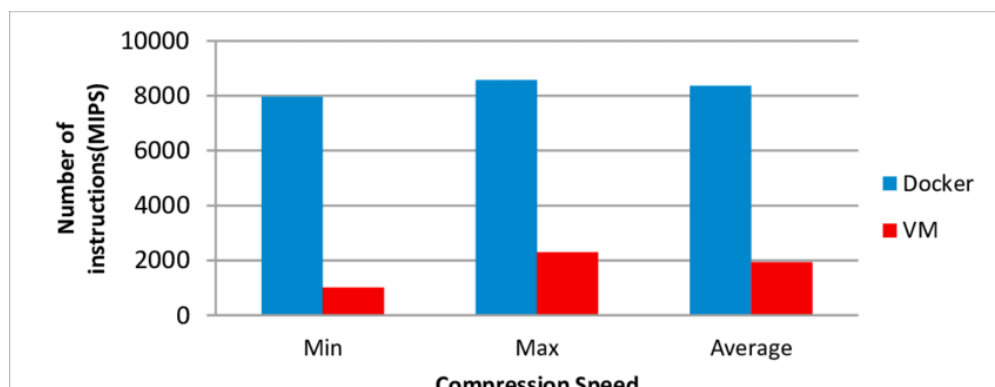


Рисунок 3 — Сравнение скорости выполнения инструкций у Docker-контейнера и виртуальной машины [5]

При этом Docker-контейнер обеспечивает достаточный уровень изоляции для данной задачи: сетевой доступ можно полностью отключить, объём памяти ограничить через механизм Linux cgroups, а время работы контролировать через тайм-аут. Ещё одно практическое преимущество — простота добавления поддержки нового языка программирования: достаточно написать Dockerfile с нужной средой выполнения и опубликовать собранный образ. Код остальных компонентов при этом не меняется.

По совокупности причин для изоляции пользовательского кода я решил использовать Docker-контейнеры.

2.3 Бизнес процесс проверки решения

В процессе проверки задействованы три участника: обучающийся, система приёма решений и исполнитель. Диаграмма бизнес-процесса в нотации BPMN приведена на рисунке 4.

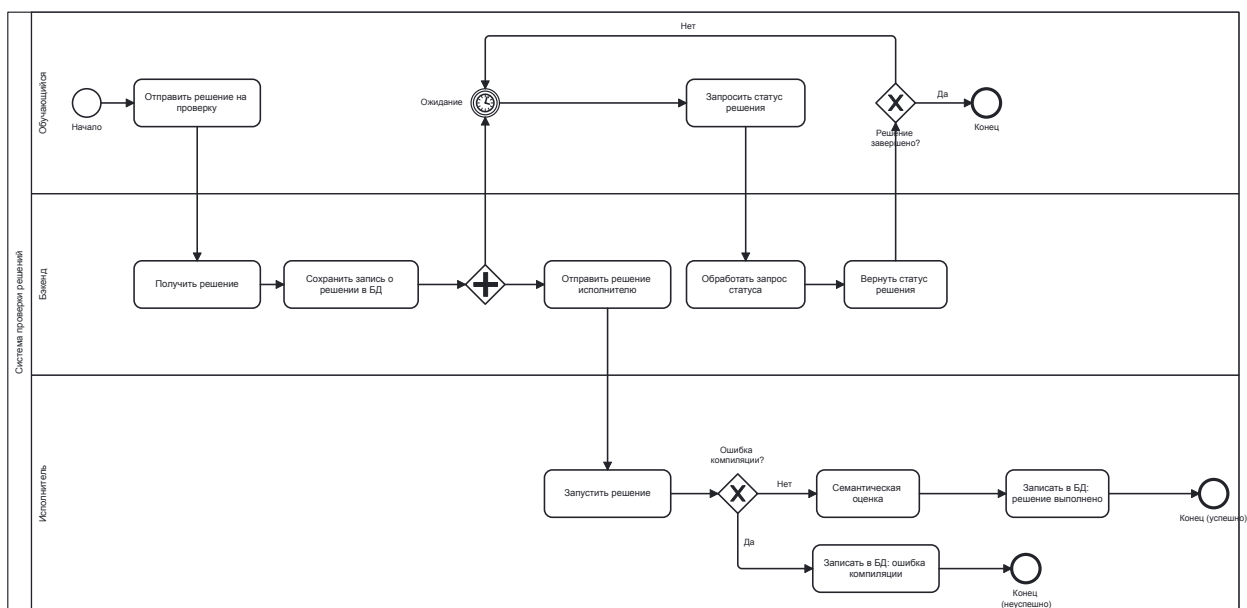


Рисунок 4 — Бизнес-процесс проверки решения

Процесс начинается с того, что обучающийся отправляет своё решение задачи. Система приёма решений фиксирует факт подачи и параллельно запускает два независимых потока: один передаёт решение на проверку исполнителю, другой — позволяет обучающемуся отслеживать ход выполнения.

Исполнитель поочерёдно проверяет решение на каждом тесте, подготовленном преподавателем. Для каждого теста он запускает программу и проверяет, завершилась ли она корректно. Если при выполнении возникла ошибка, тест фиксируется как непройденный. Если программа отработала без ошибок, над решением дополнительно производится оценка соответствия структурным ограничениям, заданным преподавателем. По завершении всех тестов исполнитель формирует итоговый результат проверки: решение засчитывается, принимается частично или отклоняется в зависимости от числа пройденных тестов.

Пока исполнитель проводит проверку, обучающийся периодически запрашивает актуальный статус своего решения. Как только все тесты завершены и итоговый результат сформирован, обучающийся получает окончательный ответ и процесс завершается.

3 Разработка системы автоматизированной проверки задач

3.1 Разработка схемы базы данных

Схема базы данных разрабатывалась исходя из требований предметной области и охватывает все основные сущности системы.

Таблица `user` хранит учётные записи пользователей: псевдоним, имя, фамилию, адрес электронной почты и хэш пароля. Поле `role` принимает одно из трёх значений перечисляемого типа: `learner` (обучающийся), `teacher` (преподаватель) или `admin` (администратор).

Таблица `contest` представляет контекст — соревнование или лабораторную работу. Его создаёт преподаватель; контекст может быть приватным или публичным, иметь общий временной лимит для участников (`time_limit`) и дату окончания приёма решений (`expires_at`).

Таблица `task` содержит задачи в составе контекста. Поле `success_threshold` определяет минимальное число успешных тестов, при котором задача засчитывается обучающемуся. Если поле не задано, требуется прохождение всех тестов.

Таблица `test` хранит отдельные тесты для задачи, подготовленные преподавателем. Каждый тест содержит входные данные (`provided_input`) и ожидаемый вывод (`expected_output`), а также ограничения по времени (`time_limit_s`) и памяти (`ram_limit_kb`). Преподаватель может отдельно разрешить или скрыть отображение входных и выходных данных теста от обучающегося.

Таблица `attempt` фиксирует попытку решения задачи: исходный код, выбранный язык программирования и текущий статус. Хэш SHA-256 исходного кода вычисляется автоматически на уровне базы данных и используется для обнаружения дублирующихся попыток: если обучающийся отправит тот же код повторно, система вернёт идентификатор уже существующей попытки.

При создании записи в таблице `attempt` срабатывает PostgreSQL-триггер `initialize_attempt`. Он создаёт запись `test_result` для каждого теста задачи и отправляет идентификатор каждой такой записи в очередь PGMQ. Бэкенду достаточно выполнить `INSERT` в таблицу `attempt` — постановка заданий в очередь производится самой базой данных.

Таблица `test_result` хранит результат выполнения конкретного теста: статус, код завершения процесса, вывод программы в `stdout` и `stderr`. Возможные статусы: `success` (тест пройден), `failure` (программа завершилась нормально, но вывод не совпал), `compfail` (ошибка компиляции или ненулевой код завершения), `timeout` (превышен временной лимит), `oom` (нехватка памяти). После обновления каждой записи в `test_result` срабатывает триггер `finalize_attempt_result`, который проверяет, завершились ли уже все тесты данной попытки, и если да — вычисляет и записывает итоговый статус попытки.

Таблица `prog_lang` содержит список поддерживаемых языков программирования с указанием Docker-образа, применяемого при проверке решений.

Схема базы данных приведена на рисунке 5.

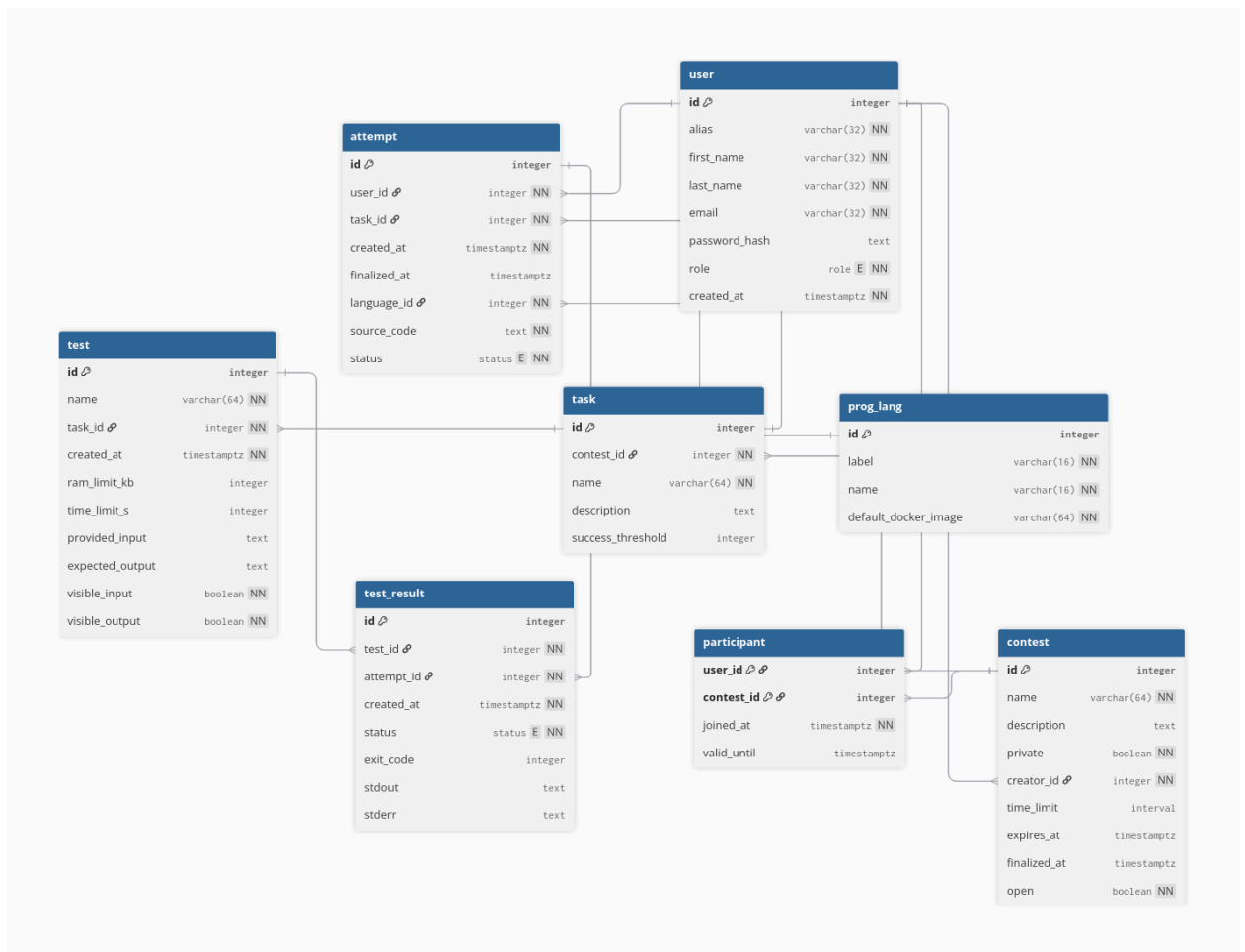


Рисунок 5 — Схема базы данных

В схеме используются четыре триггера. `initialize_attempt` срабатывает после каждой вставки в таблицу `attempt` и выполняет сразу две задачи: создаёт запись `test_result` для каждого теста задачи и отправляет идентификатор каждой такой записи в очередь PGMQ. Благодаря этому бэкенду достаточно одного INSERT — вся дальнейшая подготовка к проверке происходит на стороне базы данных. Триггер `finalize_attempt_result` запускается после каждого обновления строки в `test_result`. Он проверяет, остались ли в попытке записи со статусом `pending`; если нет — вычисляет итоговый статус попытки (`success`, `partial` или `failure`) в зависимости от числа пройденных тестов и значения `success_threshold`, после чего обновляет строку в таблице `attempt`. Триггер `set_participant_valid_until` срабатывает перед добавлением участника в контекст и автоматически вычисляет дату окончания доступа, прибавляя `time_limit` контекста к моменту присоединения. Наконец,

`refresh_views_on_participant` обновляет рейтинговые представления при каждом новом добавлении участника, поскольку его появление влияет на расчёт мест.

Помимо основных таблиц, схема включает два материализованных представления: `task_rating` и `contest_rating`. Первое хранит для каждой пары «участник — задача» время первого успешного решения и место среди всех участников конкурса. Второе агрегирует эти данные до уровня конкурса: число решённых задач и итоговое место.

Для хранения рейтингов выбраны именно материализованные представления, а не обычные. Обычное представление пересчитывается каждый раз при запросе: для рейтинговых запросов, затрагивающих все попытки всех участников конкурса, это было бы заметно медленнее. Материализованное представление ведёт себя как таблица — данные читаются напрямую, без повторных вычислений. Пересчёт происходит только в двух случаях: когда обновляется результат попытки и когда новый участник вступает в конкурс. Таким образом, дорогостоящий запрос выполняется один раз при записи, а не при каждом чтении рейтинга.

3.2 Разработка фронтенда

Фронтенд разработан с использованием библиотеки компонентов `shadcn/ui`. В отличие от обычных UI-библиотек, `shadcn/ui` предоставляет компоненты не как npm-зависимость, а в виде исходного кода, который размещается непосредственно в проекте. Это даёт полный контроль над их внешним видом и поведением: любой компонент можно изменить или доработать без необходимости переопределять стили через сложные CSS-переменные. Компоненты написаны поверх Radix UI и стилизованы с помощью Tailwind CSS.

Tailwind CSS версии 4 используется для стилизации всего приложения. Утилитарный подход позволяет описывать внешний вид прямо в JSX-разметке, что сокращает количество переключений между файлами при

разработке и исключает необходимость придумывать имена для CSS-классов.

Навигация между страницами реализована через React Router 7. Структура маршрутов отражает логику предметной области: `/contests` — список доступных контестов, `/contests/:id/tasks/:taskId` — страница конкретной задачи с редактором кода и историей попыток.

Формы строятся с использованием `react-hook-form` и библиотеки валидации `zod`. Схема данных описывается один раз и используется одновременно для проверки ввода и для автоматической типизации TypeScript.

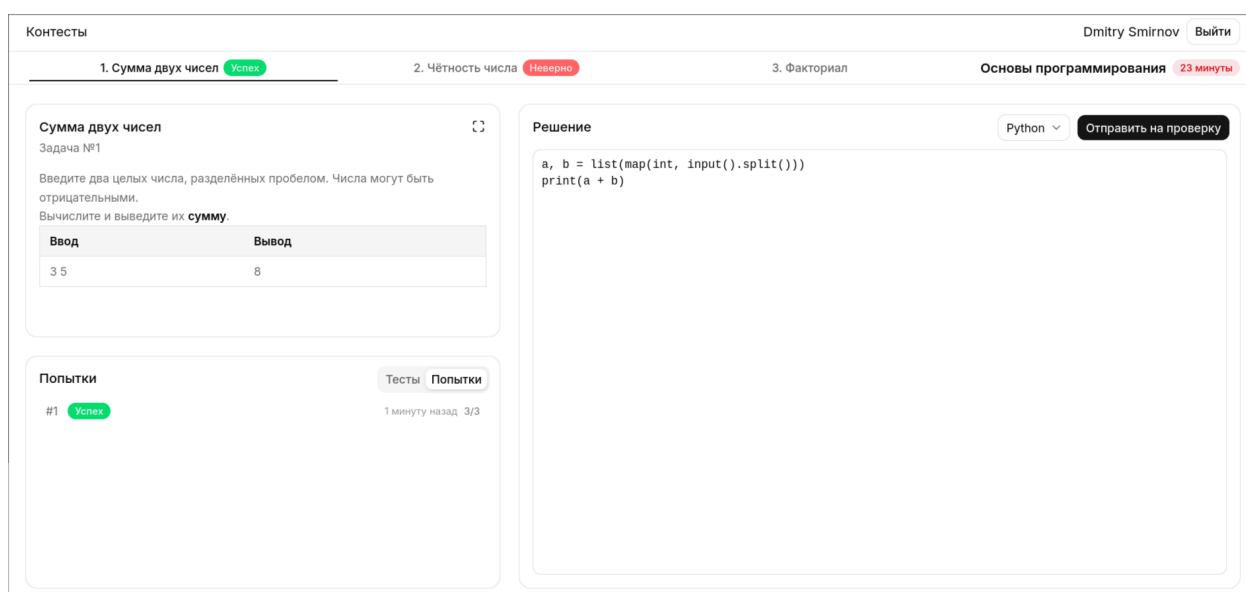


Рисунок 6 — Интерфейс страницы задачи

На рисунке 6 показан интерфейс страницы задачи для обучающегося. Слева расположено условие с примером ввода/вывода и список предыдущих попыток с их статусами. Справа находится редактор кода с выбором языка программирования и кнопкой отправки на проверку. В верхней части страницы отображаются вкладки со всеми задачами текущего контеста и оставшееся время.

Для преподавателя страница той же задачи выглядит иначе: редактор кода и кнопка отправки решения отсутствуют. В заголовке задачи появляется кнопка редактирования, позволяющая изменить условие или

параметры в любой момент. Правая панель переключается между вкладками «Попытки» и «Тесты». Вкладка «Попытки» показывает решения всех участников конкурса с их статусами и количеством пройденных тестов, что позволяет преподавателю отслеживать прогресс группы. Во вкладке «Тесты» отображается список добавленных тест-кейсов и кнопка создания нового.

При добавлении теста преподаватель выбирает метод проверки. Первый вариант — сравнение вывода программы с эталонным ответом: задаются входные данные и ожидаемый вывод, которые раннер использует при запуске решения в Docker-контейнере. Второй вариант — семантический анализ структуры кода: преподаватель описывает ограничения, которым программа должна удовлетворять, например запрет вложенных циклов или рекурсии. Такие ограничения проверяются статически, без запуска кода.

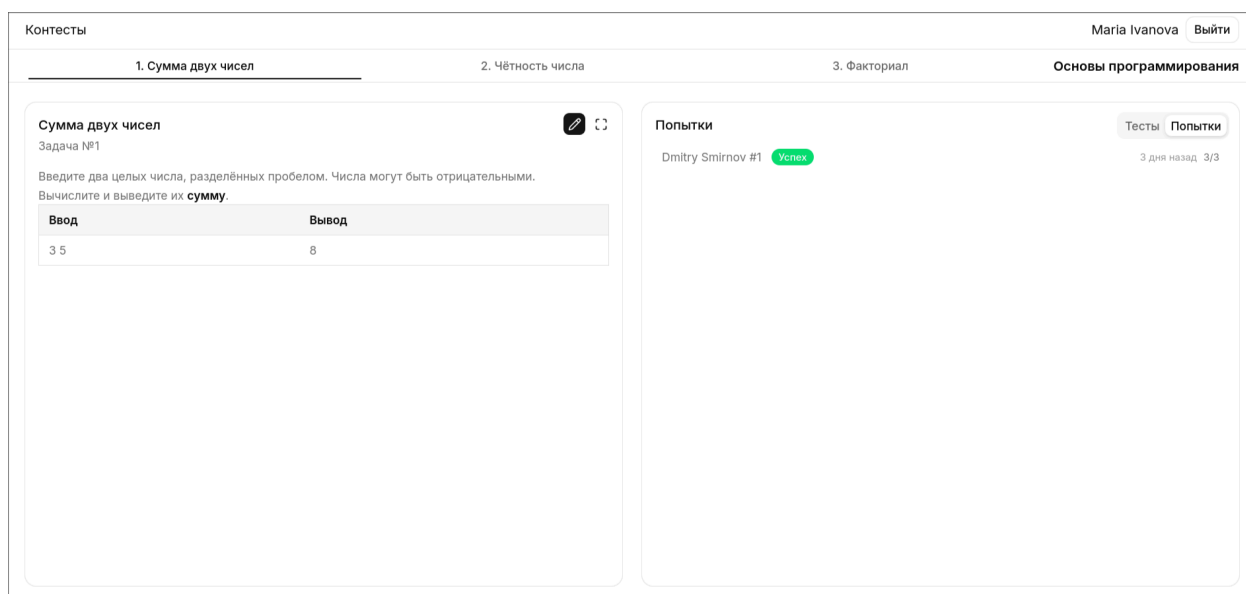


Рисунок 7 — Интерфейс страницы задачи для преподавателя

3.3 Разработка бэкенда

Бэкенд реализован на языке Rust с использованием веб-фреймворка `axum`. Приложение предоставляет REST API: запросы и ответы передаются в формате JSON. Маршруты организованы в иерархическую структуру: `/auth` — аутентификация, `/users` — управление пользователями, `/contests` — конкурсы и вложенные в них задачи, тесты и попытки, `/languages` — список доступных языков программирования.

Пользователь может иметь одну из трёх ролей: `learner`, `teacher` или `admin`. Роль определяет, какие операции доступны. Обучающийся видит открытые для него конкурсы, отправляет решения задач и получает результаты своих попыток. Преподаватель создаёт конкурсы, добавляет задачи и тесты, а также может просматривать решения участников своих конкурсов. Администратор имеет доступ ко всем ресурсам системы без ограничений.

Аутентификация построена на JWT-токенах. При входе клиент получает подписанный токен, содержащий идентификатор пользователя и его роль. Токен передаётся в заголовке `Authorization` каждого последующего запроса. Middleware на стороне сервера проверяет подпись токена и добавляет данные о пользователе к контексту запроса. Проверка прав доступа встроена непосредственно в SQL-запросы через условия на роль и идентификатор пользователя.

Большинство обработчиков выполняют один или несколько SQL-запросов к PostgreSQL через `sqlx` и возвращают результат в виде JSON. При отправке нового решения бэкенд выполняет `INSERT` в таблицу `attempt`. Дальнейшую работу берёт на себя база данных: триггер `initialize_attempt` создаёт записи `test_result` и отправляет их идентификаторы в очередь `PGMQ`. Таким образом, бэкенду не нужно взаимодействовать с очередью напрямую.

Перечень реализованных эндпоинтов REST API с указанием необходимых прав доступа приведён в таблице 1.

Таблица 1 — Реализованные эндпоинты REST API

Метод	Путь	Доступ	Описание
POST	/auth/login	Публ.	Аутентификация, возвращает JWT-токен
POST	/auth/register	Публ.	Регистрация нового пользователя
GET	/users/me	Авт.	Данные текущего авторизованного пользователя
GET	/users/me/contests	Авт.	Контеcты, в которых участвует или которые создал пользователь
GET	/users	Адм.	Список пользователей системы с фильтрацией
PUT	/users/{id}	Адм.	Обновление данных пользователя и смена роли
POST	/contests	Преп.	Создание нового контеcта
GET	/contests	Публ.	Список публичных и доступных контеcтов
GET	/contests/{id}	Публ.	Данные конкретного контеcта
PUT	/contests/{id}	Преп.	Обновление параметров контеcта
POST	/contests/{id}/join	Авт.	Вступление в контеcт
GET	/contests/{id}/rating	Авт.	Рейтинговая таблица участников
GET	/contests/{id}/progress	Авт.	Прогресс текущего пользователя по задачам
GET	/contests/{id}/participants	Преп.	Список участников с поиском по имени и email
POST	/contests/{id}/tasks	Преп.	Добавление задачи в контеcт
GET	/contests/{id}/tasks	Авт.	Список задач контеcта

Метод	Путь	Доступ	Описание
GET	/contests/{id}/tasks/{id}	Авт.	Данные конкретной задачи
PUT	/contests/{id}/tasks/{id}	Преп.	Обновление условия и параметров задачи
POST	/contests/{id}/tasks/{id}/tests	Преп.	Создание теста для задачи
GET	/contests/{id}/tasks/{id}/tests	Авт.	Список тестов задачи
PUT	/contests/{id}/tasks/{id}/tests/{id}	Преп.	Обновление теста
DELETE	/contests/{id}/tasks/{id}/tests/{id}	Преп.	Удаление теста
POST	/contests/{id}/tasks/{id}/attempts	Авт.	Отправка решения на проверку
GET	/contests/{id}/tasks/{id}/attempts	Авт.	Список попыток по задаче
GET	/contests/{id}/tasks/{id}/attempts/{id}/tests	Авт.	Результаты тестов конкретной попытки
GET	/languages	Авт.	Список поддерживаемых языков программирования

3.4 Разработка раннера решений

Раннер — отдельный сервис, также реализованный на Rust. Его задача — извлекать задания из очереди PGMQ и выполнять проверку пользовательских решений.

Сервис работает в непрерывном цикле и вызывает `pgmq.pop('test_execution')`. При наличии сообщения в очереди раннер получает из него идентификатор записи `test_result` и исходный код решения. Если очередь пуста, сервис ждёт одну секунду перед следующим запросом.

Получив задание, раннер запрашивает из базы данных параметры соответствующего теста: входные данные, ожидаемый вывод, ограничения по памяти и времени, а также имя Docker-образа для нужного языка программирования. Исходный код записывается во временный файл на диске хост-системы.

Затем с помощью библиотеки `bollard` — Rust-клиента Docker API — создаётся контейнер. Сетевой доступ для него полностью отключён, ограничение памяти задаётся через Linux `cgroups` на уровне Docker, временной лимит передаётся внутрь контейнера через переменную окружения. Файл с исходным кодом монтируется в контейнер как том, доступный только для чтения.

После запуска контейнера раннер читает его `stdout` и `stderr`. По завершении работы проверяется код завершения процесса. Коды 124 и 137 соответствуют прерыванию по тайм-ауту — в этом случае выставляется статус `timeout`. В остальных случаях содержимое `stdout` сравнивается с ожидаемым выводом после удаления пробельных символов по краям строк. При совпадении выставляется статус `success`; если программа завершилась с кодом 0, но вывод не совпал, — `failure`; при ненулевом коде завершения без тайм-аута — `compfail`, что обычно свидетельствует об ошибке компиляции.

Результат записывается в таблицу `test_result`, после чего контейнер удаляется. Триггер `finalize_attempt_result` при этом автоматически проверяет, все ли тесты попытки завершены, и при необходимости вычисляет итоговый статус.

Для каждого поддерживаемого языка программирования подготовлен отдельный Docker-образ. Он содержит компилятор или интерпретатор языка и скрипт-обёртку, которая принимает путь к файлу с исходным кодом, при необходимости компилирует его и запускает программу, пробрасывая входные данные через стандартный поток ввода. Временной лимит реализован через встроенную в скрипт утилиту `timeout`: при его превышении процесс завершается с кодом 124, который раннер

интерпретирует как статус timeout. Такой подход делает добавление нового языка полностью независимым от кода остальных компонентов системы — достаточно написать Dockerfile и скрипт запуска.

Схема прохождения решения через Docker-контейнер приведена на рисунке 8.

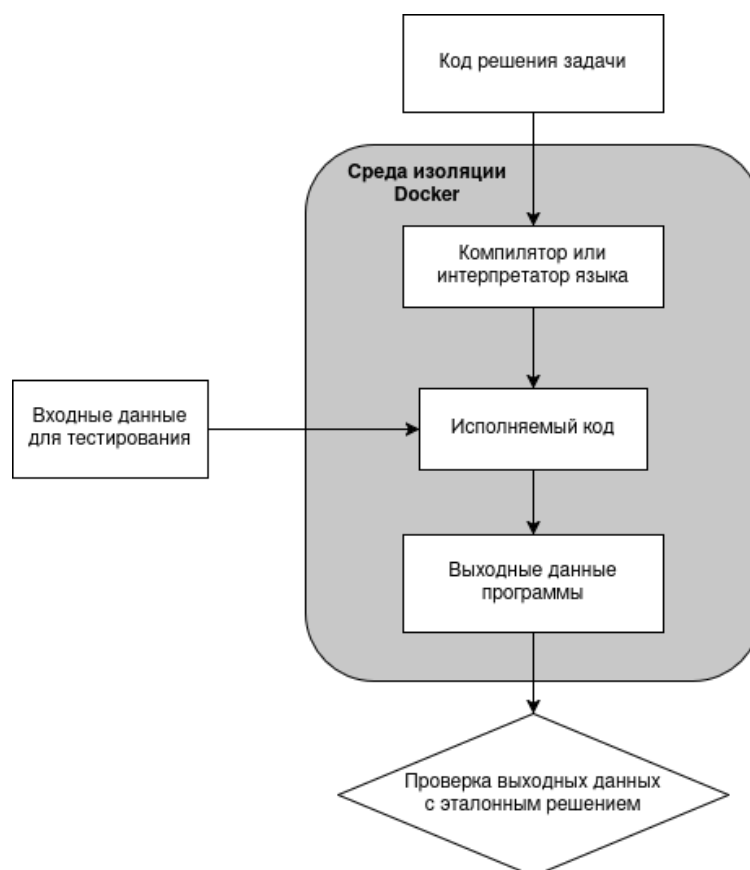


Рисунок 8 — Схема изоляции и выполнения пользовательского кода

3.5 Тестирование

Тестирование бэкенда проводилось с применением библиотеки `axum-test`. Она позволяет создать тестовый HTTP-сервер на основе существующего `axum`-приложения и выполнять к нему запросы без поднятия реального сетевого стека. Каждый тест работает с отдельной тестовой базой данных, которая инициализируется перед запуском набора тестов.

Пример unit-теста для одного из эндпоинтов я привёл на рисунке 9. Данный конкретный тест удостоверяется в том что обучающийся не сможет сам создавать новые контесты.

```

#[tokio::test]
async fn test_learner_cannot_create_contest() {
    let server = build_server().await;
    let token = obtain_token(&server, "learner@example.com").await;

    let resp = server
        .post("/contests")
        .add_header(
            "Authorization",
            format!("Bearer {token}").as_str(),
        )
        .json(&json!({ "name": "Test Contest", "private": true }))
        .await;
    resp.assert_status(StatusCode::FORBIDDEN);
}

```

Рисунок 9 — Пример unit-теста проверки разграничения прав доступа

Тестирование охватило несколько групп сценариев. Прежде всего проверялась корректность аутентификации: вход с верными и неверными учётными данными, а также поведение защищённых эндпоинтов при запросе без токена. Для ролевого разграничения прав тестировалось, что обучающийся не может создавать конкурсы и задачи, а преподаватель получает доступ только к ресурсам собственных конкурсов. Отдельно проверялась логика работы с попытками: система должна возвращать идентификатор уже существующей попытки при повторной отправке идентичного исходного кода вместо создания дублирующей записи. Также тестировалась видимость результатов тестов — обучающийся видит входные данные и вывод программы только для тех тестов, для которых преподаватель явно разрешил их отображение.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была разработана система автоматизированной проверки задач, решаемых на языках программирования. Система построена на микросервисной архитектуре и включает три независимых сервиса — бэкенд, фронтенд и раннер, — взаимодействующих через общую базу данных PostgreSQL.

В аналитической части были рассмотрены три подхода к оценке решений: сравнение вывода с эталонным ответом, применение языковых моделей и семантический анализ структуры кода. В разработанной системе были использованы методы сравнения вывода с эталонным результатом и метод семантического анализа исходного кода так как они являются объективными и предсказуемыми в условиях автоматической проверки. Метод оценки при помощи искусственного интеллекта не был задействован из-за недостаточной точности.

Безопасное исполнение пользовательского кода обеспечивается за счёт запуска каждого решения в Docker-контейнере с отключённым сетевым доступом и ограничениями по памяти. Бонусом данного подхода является то что добавление поддержки нового языка программирования не требует изменений в коде системы — достаточно подготовить соответствующий Docker-образ, чем может заняться даже преподаватель.

Взаимодействие между бэкендом и сервисами тест раннеров организовано через очередь RGMQ, надстроенную над базой данных Postgres, что позволяет масштабировать количество раннеров горизонтально без изменения архитектуры остальных компонентов.

Разработанная система использует два метода оценки решения, предоставляет преподавателям инструмент для создания контестов с задачами и тестами, мониторинга успеваемости обучающихся, а обучающимся — возможность решать задачи контестов на различных языках программирования, получая детальный отчёт по каждой попытке.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Karsa Z.I., Goldschmidt B. Automatic evaluation of programming tasks supported by language models // IEEE Access. IEEE, 2025.
2. Федоров А.С., Авксентьева Е.Ю. Алгоритм оценки качества решения задач по программированию // Вестник Астраханского государственного технического университета. Серия: Управление, вычислительная техника и информатика. 2024. № 1. Сс. 100–108.
3. Мальцев В.М., Назаров И.А., Павлова Е.А. Исследование применимости генеративной нейронной сети в качестве классификатора решений заданий по программированию // Математика. Информатика. Информационные технологии. Математическое моделирование. 2024. Сс. 92–98.
4. Rust Adoption Drives Android Memory. <https://thehackernews.com/2025/11/rust-adoption-drives-android-memory.html> Дата обращения: 17.03.2026.
5. Potdar A.M. и др. Performance Evaluation of Docker Container and Virtual Machine // Procedia Computer Science. Elsevier, 2020. Т. 171. Сс. 1419–1428.